```
<#
  HELO-probe.ps1 - find the REST parameter IDs for Recording Template, SMB Server
  SMB Network Record Path on a HELO.

  This script only READS (HTTP "get"). YOU make the changes on the unit's web pag
  Use ONE idle spare HELO (not recording), and put its settings back at the end.

  Usage:  powershell -ExecutionPolicy Bypass -File .\HELO-probe.ps1 -Ip 192.168.x

  1. Collects every eParamID_* name it can find (the web page's own scripts + a c
  2. Reads them all (baseline x2, so values that change by themselves are ignored
  3. Walks you through: change template -> change template back -> change SMB ser
     After each step it shows which parameters changed.
  Results are saved to helo-probe-*.txt next to the script. Copy the on-screen ou
#>
param(
    [Parameter(Mandatory = $true)][string]$Ip,
    [string]$OutDir
)
if ($Ip -notmatch '^\d{1,3}(\.\d{1,3}){3}$') { Write-Warning "Not a valid IP: $Ip
if (-not $OutDir) { $OutDir = if ($PSScriptRoot) { $PSScriptRoot } else { (Get-Lo

Add-Type -AssemblyName System.Net.Http
$h = New-Object System.Net.Http.HttpClientHandler
$h.UseProxy = $false
$client = New-Object System.Net.Http.HttpClient($h)
$client.Timeout = [TimeSpan]::FromSeconds(4)
$client.DefaultRequestHeaders.ConnectionClose = $true

function Get-Raw($path) {
    try   { return $client.GetStringAsync("http://$Ip$path").GetAwaiter().GetResu
    catch { return $null }
}

# ---- 1. collect parameter names ----
$names = @{}
function Add-Names($text) {
    if (-not $text) { return }
    foreach ($m in [regex]::Matches($text, 'eParamID_[A-Za-z0-9_]+')) { $names[$m
}

Write-Host "HELO at $Ip  (read-only)"
Write-Host 'Looking for parameter names...'
$seen = @{}
```

```
$queue = New-Object System.Collections.Queue
$queue.Enqueue('/')
while ($queue.Count -gt 0 -and $seen.Count -lt 60) {
    $p = [string]$queue.Dequeue()
    if ($seen.ContainsKey($p)) { continue }
    $seen[$p] = $true
    $body = Get-Raw $p
    if (-not $body) { continue }
    Add-Names $body
    foreach ($m in [regex]::Matches($body, '(?:src|href)\s*=\s*["'']([^"'']+\.(?:
        $u = $m.Groups[1].Value
        if ($u -match '^https?://') { continue }            # only this unit
        if (-not $u.StartsWith('/')) { $u = '/' + $u }
        if (-not $seen.ContainsKey($u)) { $queue.Enqueue($u) }
    }
}
Write-Host ("  from the web page files: {0} names (pages/scripts read: {1})" -f $

$dump = Get-Raw '/config?action=get&configid=0'
if ($dump) { $before = $names.Count; Add-Names $dump; Write-Host ("  from a confi

# keep names we know are commands out of the polling list (a "get" is harmless, b
$list = @($names.Keys | Where-Object { $_ -notmatch 'Command$' } | Sort-Object)
if ($list.Count -eq 0) {
    Write-Warning 'No parameter names found. Use the browser: F12 > Network, clic
    return
}
Write-Host ("Will read {0} parameters each time." -f $list.Count)

function Get-Snapshot {
    $map = @{}
    $i = 0
    foreach ($n in $list) {
        $i++
        if ($i % 25 -eq 0) { Write-Host -NoNewline '.' }
        $raw = Get-Raw "/config?action=get&paramid=$n"
        if (-not $raw) { continue }
        try {
            $j = $raw | ConvertFrom-Json
            $v = "$($j.value)"
            if ($j.value_name) { $v += " ($($j.value_name))" }
            $map[$n] = $v
        } catch { $map[$n] = $raw.Trim() }
    }
    Write-Host ''
    return $map
}
```

```
function Save-Snapshot($label, $map) {
    $lines = $map.GetEnumerator() | Sort-Object Name | ForEach-Object { '{0} = {1
    [System.IO.File]::WriteAllLines((Join-Path $OutDir "helo-probe-$label.txt"),
}
function Show-Diff($old, $new, $noisy) {
    $any = $false
    foreach ($k in (@($old.Keys) + @($new.Keys) | Sort-Object -Unique)) {
        if ($noisy.ContainsKey($k)) { continue }
        if ("$($old[$k])" -ne "$($new[$k])") {
            $any = $true
            Write-Host ('  {0}:  {1}  ->  {2}' -f $k, $(if ($null -eq $old[$k]) {
        }
    }
    if (-not $any) { Write-Host '  (nothing changed - did Apply/Save get pressed?
}

# ---- 2. baseline ----
[void](Read-Host "`nMake sure this HELO is NOT recording. Note its current templa
$a = Get-Snapshot
Start-Sleep -Seconds 2
$b = Get-Snapshot
if ($a.Count -eq 0) { Write-Warning "Could not read any parameter from $Ip."; ret
$noisy = @{}
foreach ($k in (@($a.Keys) + @($b.Keys))) { if ("$($a[$k])" -ne "$($b[$k])") { $n
Write-Host ("Read {0} values; ignoring {1} that change on their own." -f $a.Count
Save-Snapshot 'baseline' $a

# ---- 3. guided changes ----
$steps = @(
    @('template-changed', 'On the unit''s web page change the RECORDING TEMPLATE
    @('template-1080low', 'Now change the template to "1080p Low 5 Mbps" and clic
    @('smb-server',       'Now change the SMB SERVER ADDRESS slightly (e.g. add a
    @('smb-path',         'Now change the SMB NETWORK RECORD PATH slightly (e.g.
)
$prev = $a
foreach ($s in $steps) {
    $ans = Read-Host ("`n{0}`n  then press Enter   [q = stop]" -f $s[1])
    if ($ans -match '^(q|quit)$') { break }
    $snap = Get-Snapshot
    Save-Snapshot $s[0] $snap
    Write-Host ("Changed since the previous step ({0}):" -f $s[0])
    Show-Diff $prev $snap $noisy
    $prev = $snap
}

Write-Host "`nNow PUT THE SMB SERVER AND PATH BACK to their original values on th
$client.Dispose()
```

```
Write-Host "Done. Copy everything above and send it to me (files are in $OutDir).
```